Sensor Fusion for Mobile-Robot Localization

Hybrid Kalman Filter and Artificial Neural Network Scheme
on the Pioneer P3-DX (CoppeliaSim)

Robotics Project — Politecnico di Torino

Hussein Ghaddar Fatima Zouhour Rawan Elhalabi Maya Kawssan

Student IDs: _____ _____ _____ _____

June 4, 2026

Abstract

We implement and study a multi-sensor localization scheme for a differential-drive mobile robot, based on the method of Yousuf & Kadri (*Robotica*, 2020). The robot is simulated in CoppeliaSim. Two Extended Kalman Filters fuse GPS with inertial (KF-1) and odometric (KF-2) data; their outputs are combined by a complementary filter. A feed-forward neural network is trained, while GPS is available, to act as a pseudo-GPS during signal outages, and its prediction is blended with KF-2 to recover position when GPS is lost. This report describes the architecture, derives the filter equations we implemented, presents results, and discusses the simplifications we made relative to the original paper.

Contents

1 Introduction

Accurate self-localization is fundamental for autonomous mobile robots. No single sensor is sufficient: GPS is accurate but unavailable indoors and in tunnels; inertial (INS) and odometric dead reckoning are always available but drift over time through integration error and, for odometry, wheel slippage. Fusing complementary sensors gives a better estimate than any one alone.

This project reproduces, in a course setting, the localization scheme of Yousuf & Kadri (2020). The goal is to demonstrate understanding of (i) Extended Kalman filtering for nonlinear motion models, (ii) complementary multi-sensor fusion, and (iii) the use of a neural network as a learned GPS substitute during outages. We do not attempt a full reproduction of the paper; the scope and our simplifications are stated explicitly in Section 6.

2 System Architecture

The pipeline is a chain of MATLAB scripts that pass data through `.mat` files:

1. `BaseMatlab.m` — drives the Pioneer P3-DX in CoppeliaSim and logs ground truth, GPS, IMU, odometry and a dead-reckoned INS.
2. `ekf_gps_ins.m` — 5-state EKF fusing GPS and INS (KF-1).
3. `ekf_gps_odo.m` — 3-state EKF fusing GPS and odometry (KF-2).
4. `ekf_fusion.m` — complementary fusion of KF-1 and KF-2.
5. `RobotANN_Class.m` — the feed-forward neural network.
6. `ekf_ANN_fusion.m` — trains the ANN and blends it with KF-2 during the GPS outage.

The scripts must be run in this order, since each consumes the previous one's output. A single runner script is given in Appendix A.

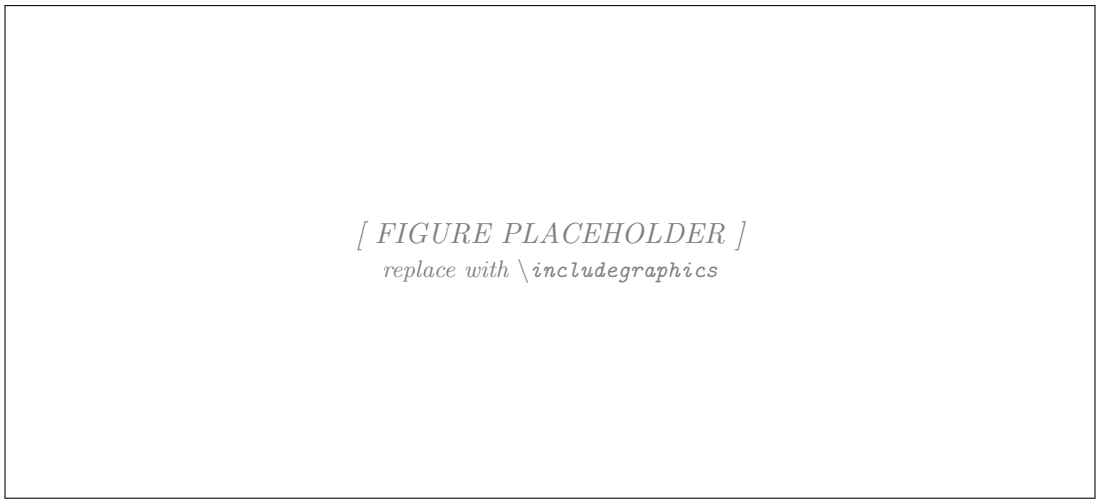

Figure 1: Block diagram of the fusion architecture: sensors feed KF-1 and KF-2; their outputs are combined by the complementary filter while GPS is available; the ANN, trained on this data, replaces GPS during an outage. (Redraw adapting the paper's Fig. 1.)

3 Kalman Filter Models

Each EKF runs the standard predict–correct cycle: the state is propagated through the nonlinear motion model, then corrected by the GPS measurement *only when GPS is available*. During an outage the predicted state is carried forward (dead reckoning). Both filters use the numerically stable *Joseph form* for the covariance update and re-symmetrise P each step.

3.1 KF-1: GPS + INS

State $\mathbf{x} = [x,\, y,\, v_x,\, v_y,\, \theta]^\top$. Body-frame accelerations are rotated into the world frame,

$$a_x^w = \cos\theta\, a_x^b - \sin\theta\, a_y^b, \qquad a_y^w = \sin\theta\, a_x^b + \cos\theta\, a_y^b,$$

and the prediction is the constant-acceleration model

$$\mathbf{x}^- = \begin{bmatrix} x + v_x\Delta t + \frac{1}{2}a_x^w\Delta t^2 \\ y + v_y\Delta t + \frac{1}{2}a_y^w\Delta t^2 \\ v_x + a_x^w\Delta t \\ v_y + a_y^w\Delta t \\ \theta + \dot{\theta}\,\Delta t \end{bmatrix}, \quad F = \frac{\partial f}{\partial \mathbf{x}} = \begin{bmatrix} 1 & 0 & \Delta t & 0 & -\frac{1}{2}\Delta t^2 a_y^w \\ 0 & 1 & 0 & \Delta t & \frac{1}{2}\Delta t^2 a_x^w \\ 0 & 0 & 1 & 0 & -\Delta t\, a_y^w \\ 0 & 0 & 0 & 1 & \Delta t\, a_x^w \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}.$$

GPS observes position only, $H = [\, I_2 \;\; 0_{2\times 3}\,]$.

3.2 KF-2: GPS + odometry

State $\mathbf{x} = [x,\, y,\, \theta]^\top$ with the unicycle model driven by forward speed v and yaw rate ω from the wheel encoders:

$$\mathbf{x}^- = \begin{bmatrix} x + v\cos\theta\,\Delta t \\ y + v\sin\theta\,\Delta t \\ \theta + \omega\,\Delta t \end{bmatrix}, \quad F = \begin{bmatrix} 1 & 0 & -v\sin\theta\,\Delta t \\ 0 & 1 & v\cos\theta\,\Delta t \\ 0 & 0 & 1 \end{bmatrix}, \quad G = \frac{\partial f}{\partial [v,\omega]} = \begin{bmatrix} \cos\theta\,\Delta t & 0 \\ \sin\theta\,\Delta t & 0 \\ 0 & \Delta t \end{bmatrix}.$$

Heading is wrapped to $(-\pi, \pi]$ with `atan2(sin,cos)` before and after each update. GPS again observes position, $H = [\, I_2 \;\; 0_{2\times 1}\,]$.

3.3 Complementary fusion

The two position estimates are combined with a constant blending coefficient $\alpha_1 \in [0, 1]$ (paper Eq. 2):

$$[x_c,\, y_c] = \alpha_1\,[x_{kf1},\, y_{kf1}] + (1 - \alpha_1)\,[x_{kf2},\, y_{kf2}].$$

We used $\alpha_1 = 0.25$.

4 Neural Network as Pseudo-GPS

While GPS is available, a feed-forward network is trained to map a 10-dimensional feature vector (acceleration and velocity magnitudes and their cumulative sums, heading and cumulative heading, and odometric velocities) to the filtered position. The architecture follows the paper: two hidden layers of 10 and 5 neurons with log-sigmoid activations, a linear output layer, quasi-Newton (`trainoss`) training, input normalization (`mapminmax`), and weight-penalty regularization.

During a GPS outage the trained network produces a position prediction $[x_n, y_n]$, which is blended with KF-2 (running on the last GPS update) by a coefficient α_{1_fuzzy} (paper Eq. 5):

$$[x_{final},\, y_{final}] = \alpha_{1\_fuzzy}\,[x_n,\, y_n] + (1 - \alpha_{1\_fuzzy})\,[x_{kf2},\, y_{kf2}].$$

In the original paper this coefficient is produced by a fuzzy logic system that reacts to wheel slip; in our implementation we used a fixed value (see Section 6).

5 Results

The figures and table below are placeholders to be filled from a single reproducible run (fixed seed `rng(0)`, one GPS outage window). Save the plots into `./figs/` and replace the placeholders.

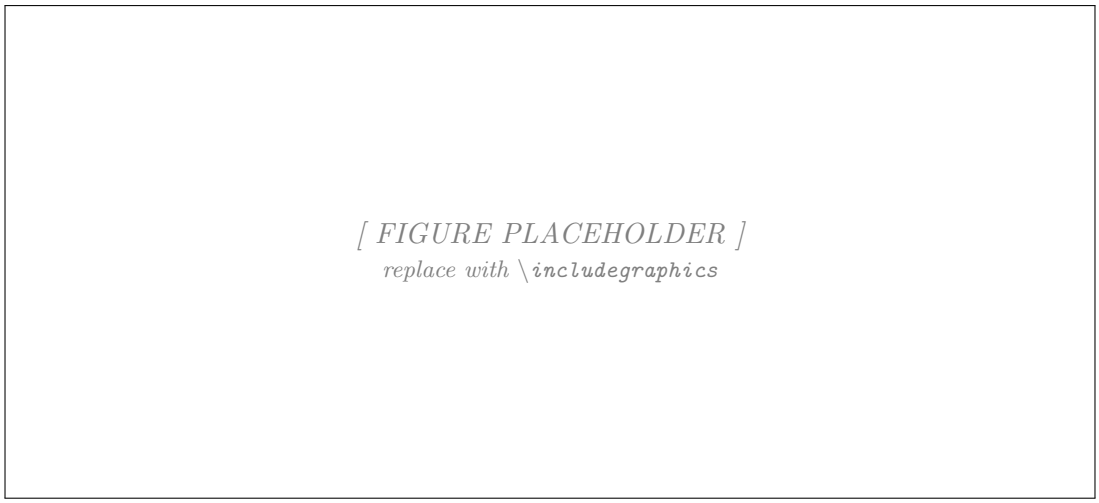

Figure 2: Raw sensor data: ground truth vs. odometry vs. raw/smoothed GPS vs. INS (from `BaseMatlab.m`).

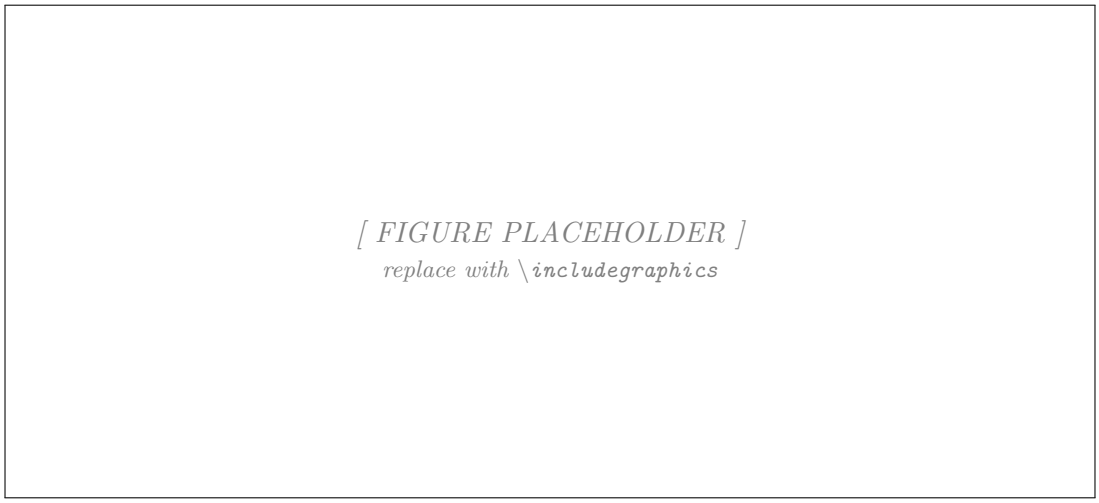

Figure 3: KF-1 (GPS-INS) estimated trajectory against ground truth.

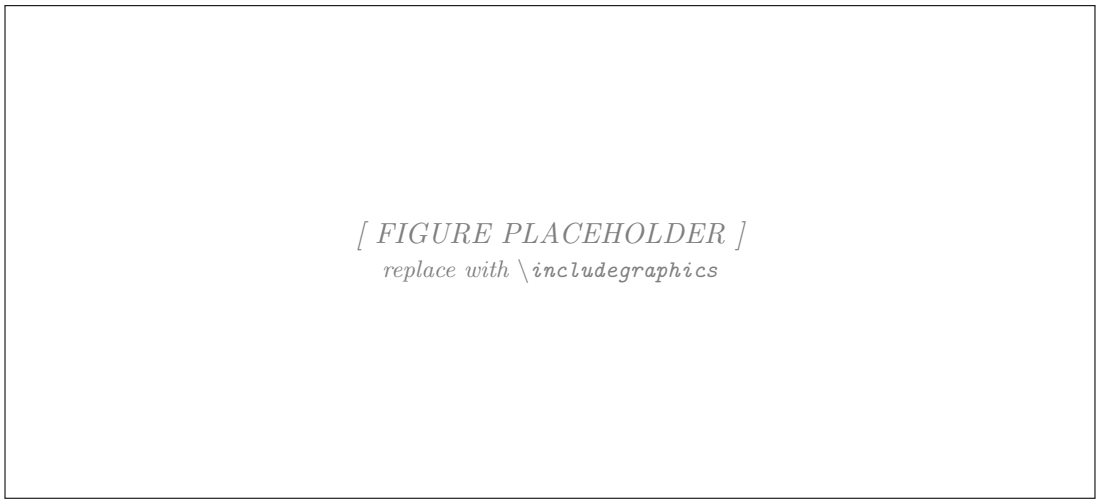

Figure 4: KF-2 (GPS-odometry) estimated trajectory against ground truth.

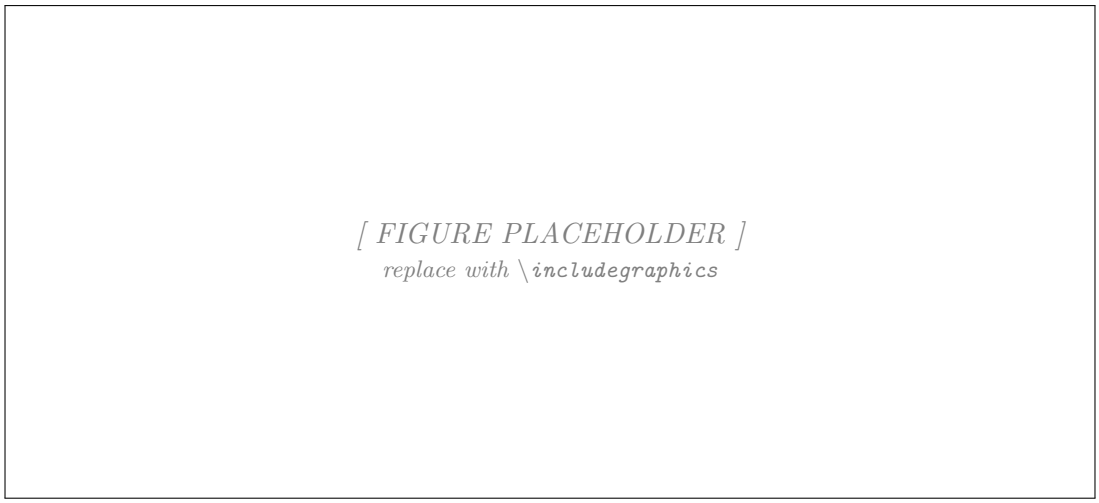

Figure 5: Final fusion during the GPS outage: truth, KF-2 alone, ANN alone, and the ANN+KF-2 blend (from `ekf_ANN_fusion.m`).

Table 1: Position RMSE (metres) — to be filled from the run.

Estimator	RMSE during outage	RMSE whole run
KF-1 (GPS-INS)	_____	_____
KF-2 (GPS-odometry)	_____	_____
Complementary fusion	_____	_____
ANN alone	_____	_____
ANN + KF-2	_____	_____

A short verification we recommend running is a heading-consistency check: plotting KF-1's heading, KF-2's heading, and the course angle from ground truth on one axis confirms that both filters and the simulator agree on the sign convention of the yaw rate. If the two filter headings rotate in opposite directions, the complementary fusion partially cancels and the sign in the INS/EKF heading update must be flipped.

6 Simplifications and Limitations

Relative to Yousuf & Kadri (2020), we made the following simplifications, which we note as honest scope decisions and possible future work:

- **Reduced filter states.** We used a 5-state KF-1 and a 3-state KF-2, rather than the 11- and 7-state formulations of the paper. Our states capture the planar pose and velocity needed for 2-D localization.
- **Fixed blending coefficients.** Both fusion stages use constant weights. A natural improvement is inverse-variance weighting for the complementary filter (the EKFs already provide their covariances), which would automatically down-weight whichever filter is more uncertain during an outage.
- **Fuzzy logic system not implemented.** The paper's fuzzy slip-detector that sets α_{1_fuzzy} was replaced by a fixed value. Implementing the three-rule Mamdani system (error between odometer and INS velocities, three triangular membership functions, centre-of-gravity defuzzification) is the main avenue for future work, and would allow the injected wheel slip to be compensated.

- **ANN training target and features.** We trained the network on the KF-2 output and used the raw wrapped heading as a feature; training on the combined KF output and encoding heading as $(\sin\theta, \cos\theta)$ would likely improve generalization.

The core filtering and fusion pipeline is functional and the EKF formulations were verified analytically. The above points are deliberate reductions in scope suited to the course, not implementation defects.

7 Conclusion

We implemented a hybrid KF/ANN localization scheme for a simulated differential-drive robot and reproduced its essential behaviour: two complementary EKFs fuse GPS with inertial and odometric data, and a trained neural network substitutes for GPS during outages. The project demonstrates the predict–correct EKF cycle on nonlinear motion models, complementary multi-sensor fusion, and the pseudo-sensor role of a neural network. Future work centres on adaptive (covariance- and fuzzy-based) blending and a richer neural feature design.

A Master run-script

Listing 1: `run_all.m` — runs the pipeline in the correct order.

```
clc; clear; close all;
run('BaseMatlab.m'); % -> kf_input.mat (CoppeliaSim must be running)
run('ekf_gps_ins.m'); % -> kf1_output.mat
run('ekf_gps_odo.m'); % -> kf2_output.mat
run('ekf_fusion.m'); % complementary fusion
run('ekf_ANN_fusion.m'); % ANN + KF-2 during outage
disp('Pipeline complete.');
```

Reference

S. Yousuf and M. B. Kadri, "Information Fusion of GPS, INS and Odometer Sensors for Improving Localization Accuracy of Mobile Robots in Indoor and Outdoor Applications," *Robotica*, 2020. doi:10.1017/S0263574720000351.